

```

    }
    else if (Y < other.Y)
    {
        return -1;
    }
    else if (Y > other.Y)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

public void Test()
{
    IntPair[] arr = new IntPair[] { new IntPair(3, 4),
new IntPair(5, 1),
new IntPair(1, 7), new
IntPair(5, 3) };
    Array.Sort(arr);
    foreach (IntPair p in arr)
    {
        Console.WriteLine("{0}, {1}", p.X, p.Y);
    }
}
}

```

Encapsulation deals with exposing a public interface to clients of a class while hiding the details of the implementation of that interface. Properties embody this principle precisely. Finally, extensibility deals with being able to extend code beyond what was originally planned. Inheritance and dynamic dispatch allow us to accomplish this. Consider the entities in a video game. To realise this in an object- oriented language, we might design an abstract Entity object.

```

public abstract class Entity
{
    public abstract void Move();
    / ...and other methods/partial implementation /
}

```